STACK • QUEUE • DEQUE

Java Ultimate Interview Handbook

Concepts → Recognition clues → Operations → Patterns → Complete Java code → Dry runs → Complexity → Interview practice

---

Is handbook ka goal: sirf definitions yaad karna nahi. Question dekhkar pehchanna hai ki Stack, Queue ya Deque kyun use hoga, kaunsa operation kab karna hai, aur Java mein khud se code kaise likhna hai.

Language: Java. Explanations simple Hinglish mein. Code interview-style, reusable aur comments ke saath.

Important: Java mein modern code ke liye **ArrayDeque** ko default choice samjho. Legacy Stack class aur LinkedList ke trade-offs bhi cover hain.

Handbook map

- 1. Stack fundamentals, LIFO, operations, Java implementations and pitfalls
- 2. Queue fundamentals, FIFO, circular queue, linked queue and priority queue distinction
- 3. Deque: both-end operations, monotonic deque and sliding-window maximum
- 4. Interview patterns: parentheses, next greater element, histogram, BFS, cache and parsing
- 5. Complete Java templates, dry runs, complexity table, mistakes and practice roadmap

1. Sabse pehle: Stack, Queue aur Deque ka mental model

Stack = LIFO (Last In, First Out): jo element sabse last mein add hua, woh sabse pehle niklega. Plates ka stack imagine karo.

Queue = FIFO (First In, First Out): jo element sabse pehle aaya, woh sabse pehle niklega. Ticket counter ki line imagine karo.

Deque = Double-Ended Queue: front aur back dono ends se insert aur remove kar sakte ho. Ye Stack aur Queue dono ke kaam aa sakta hai.

Structure	Add	Remove	Typical use
Stack	Top par push	Top se pop	Undo, brackets, DFS, monotonic stack
Queue	Rear par offer	Front se poll	BFS, scheduling, level order
Deque	Front ya rear	Front ya rear	Sliding window, 0-1 BFS, both-end tasks

Question dekhkar kaise pehchanein?

- “Most recent”, “undo”, “nested”, “matching brackets”, “next greater/smaller” → Stack ka strong signal.
- “First come first served”, “level by level”, “minimum steps in an unweighted graph” → Queue/BFS ka signal.
- “Window ke andar maximum/minimum”, “dono ends se operation”, “front aur back” → Deque ka signal.
- “Priority ke hisaab se next” → PriorityQueue/Heap; normal Queue nahi.

Golden rule: Data structure question ke words se nahi, required order of processing se choose karo.

2. Stack fundamentals

LIFO ko step-by-step samjho

Java code

```
Stack<Integer> st = new ArrayDeque<>();

st.push(10);  // bottom: 10
st.push(20);  // 20, 10
st.push(30);  // top: 30

System.out.println(st.peek()); // 30
System.out.println(st.pop());  // removes 30
System.out.println(st.pop());  // removes 20
```

Stack ko vertical pile samjho. `push(x)` top par rakhta hai, `pop()` top ko hata kar return karta hai, aur `peek()` top ko bina hataye dekhta hai.

Stack ke core operations

Operation	Meaning	Typical time
push(x)	Top par element add	O(1) amortized
pop()	Top element remove + return	O(1) amortized
peek()	Top element read, remove nahi	O(1)
isEmpty()	Stack khaali hai?	O(1)
size()	Kitne elements hain?	O(1)

Java mein Stack kaise banayein?

Recommended: `Deque` interface ke saath `ArrayDeque` use karo. `Stack` class purani (legacy) hai aur synchronized methods ki wajah se aksar interview code mein preferred nahi hoti.

Java code

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
```

```
        Deque<Integer> stack = new ArrayDeque<>();

        stack.push(5);
        stack.push(10);
        stack.push(15);

        System.out.println(stack.peek()); // 15
        System.out.println(stack.pop());  // 15
        System.out.println(stack.size()); // 2
        System.out.println(stack.isEmpty()); // false
    }
}
```

Stack: empty case aur safe access

`pop()` ya `peek()` empty stack par call karoge to `NoSuchElementException` aa sakta hai. Pehle `isEmpty()` check karo. `poll()` / `peek()` ke Deque queue-style methods empty par `null` de sakte hain; stack-style `pop()` exception deta hai.

Java code

```
if (!stack.isEmpty()) {
    int top = stack.peek();
    int removed = stack.pop();
}
```

ArrayDeque vs Stack vs LinkedList

Choice	Good points	Caution
ArrayDeque	Fast, clean, usually best default	Null elements allowed nahi
Stack	Simple old API	Legacy; extends Vector; avoid as default
LinkedList	Deque operations possible	Extra node memory; often slower/cache-unfriendly

3. Stack pattern: Balanced Parentheses

Problem signal: opening bracket ko future closing bracket ke saath match karna hai; nested brackets hain. Last opened bracket sabse pehle close hoga, isliye Stack.

Algorithm: opening bracket aaye to push. Closing bracket aaye to stack empty nahi hona chahiye aur top matching opening bracket hona chahiye. End mein stack empty hona zaroori hai.

Java code

```
import java.util.*;

class Solution {
    public boolean isValid(String s) {
        Deque<Character> st = new ArrayDeque<>();

        for (char ch : s.toCharArray()) {
            if (ch == '(' || ch == '[' || ch == '{') {
                st.push(ch);
            } else {
                if (st.isEmpty()) return false;

                char open = st.pop();
                if (ch == ')' && open != '(') return false;
                if (ch == ']' && open != '[') return false;
                if (ch == '}' && open != '{') return false;
            }
        }
        return st.isEmpty();
    }
}
```

Dry run: s = "{[()] }" (space hata kar "{[()]}")

Character	Action	Stack (top at right)
{	push {	{
[	push [	{ [
(	push (	{ [(
)	pop (, match	{ [
]	pop [, match	{
}	pop {, match	empty

Complexity: O(n) time, O(n) extra space. Har character maximum ek baar push aur ek baar pop hota hai.

Common mistakes

- Sirf final stack empty check karna enough nahi; har closing bracket ka type bhi match hona chahiye.
- Closing bracket ke waqt stack empty ho sakta hai—pehle check karo.
- Input empty string ho to valid result true hota hai (agar problem definition standard ho).

4. Stack pattern: Next Greater Element

Problem: har array element ke right side mein pehla strictly greater element find karo. Na mile to -1.

Recognition: “next greater”, “next smaller”, “previous greater/smaller” aksar Monotonic Stack ka signal hai.

Idea: indices stack mein rakho. Current value kisi previous unresolved value se badi hai to current uska next greater answer hai; pop karke answer fill karo.

Java code

```
import java.util.*;

class Solution {
    public int[] nextGreaterRight(int[] a) {
        int n = a.length;
        int[] ans = new int[n];
        Arrays.fill(ans, -1);

        Deque<Integer> st = new ArrayDeque<>(); // indices

        for (int i = 0; i < n; i++) {
            while (!st.isEmpty() && a[i] > a[st.peek()]) {
                int idx = st.pop();
                ans[idx] = a[i];
            }
            st.push(i);
        }
        return ans;
    }
}
```

Dry run: [2, 1, 3, 2, 4]

i / value	Stack before current (indices)	What happens
0 / 2	[]	push 0
1 / 1	[0]	1 is not > 2; push 1
2 / 3	[1, 0]	3 resolves 1 and 0; push 2
3 / 2	[2]	push 3
4 / 4	[3, 2]	4 resolves 3 and 2; push 4

Answer: [3, 3, 4, 4, -1]. **Complexity:** O(n) time and O(n) space. Har index ek baar push aur maximum ek baar pop hota hai.

Variants jo interview mein aate hain

- Next Smaller Element: pop while current value is smaller (comparison reverse).
- Previous Greater/Smaller: left-to-right scan; current answer stack top se mil sakta hai.
- Circular Next Greater: array ko conceptually 2 rounds scan karo; index `i % n` use karo, push sirf first round mein.
- Daily Temperatures: next warmer day; answer mein distance `i - previousIndex` store karo.

5. Stack pattern: Largest Rectangle in Histogram

Problem signal: histogram bars ki heights di hain; maximum-area rectangle. Har bar ko minimum-height bar maan kar uske left/right mein pehli chhoti height find karni hoti hai.

Monotonic increasing stack of indices use hota hai. Jab current height stack top se chhoti ho, popped bar ka right boundary current index aur left boundary new stack top hota hai.

Java code

```
import java.util.*;

class Solution {
    public int largestRectangleArea(int[] h) {
        int n = h.length;
        Deque<Integer> st = new ArrayDeque<>();
        int best = 0;

        for (int i = 0; i <= n; i++) {
            int current = (i == n) ? 0 : h[i];

            while (!st.isEmpty() && h[st.peek()] > current) {
                int height = h[st.pop()];
                int leftSmaller = st.isEmpty() ? -1 : st.peek();
                int width = i - leftSmaller - 1;
                best = Math.max(best, height * width);
            }
            st.push(i);
        }
        return best;
    }
}
```

Complexity: O(n) time, O(n) space. Note: sentinel `0` final pending bars ko flush karta hai. Heights non-negative honi chahiye; empty array ka answer 0.

Stack ke aur high-frequency problems

Problem	Core trick
Min Stack	Extra stack ya pair; minimum in O(1)
Evaluate Reverse Polish Notation	Numbers push; operator par two operands pop
Decode String	Nested repeat count + previous string stack
Asteroid Collision	Right-moving stack top vs incoming left-moving asteroid
Remove K Digits	Monotonic increasing stack; larger previous digits remove
Simplify Path	Path components stack
Stock Span	Previous greater element using monotonic stack

6. Min Stack: getMin() in O(1)

Normal stack mein minimum nikalne ke liye scan O(n) ho sakta hai. Interview version mein `push`, `pop`, `top`, `getMin` sab O(1) chahiye.

Approach: values ke saath current minimum bhi store karo. Har push par pair `(value, minSoFar)` push hota hai.

Java code

```
import java.util.*;

class MinStack {
    private Deque<int[]> st = new ArrayDeque<>();
```

```
    public void push(int val) {
        int min = st.isEmpty() ? val : Math.min(val, st.peek()[1]);
        st.push(new int[]{val, min});
    }

    public void pop() {
        if (st.isEmpty()) throw new NoSuchElementException();
        st.pop();
    }

    public int top() {
        if (st.isEmpty()) throw new NoSuchElementException();
        return st.peek()[0];
    }

    public int getMin() {
        if (st.isEmpty()) throw new NoSuchElementException();
        return st.peek()[1];
    }
}
```

Complexity: all operations O(1); extra space O(n). Alternative: two stacks (values + min values).

RPN evaluation: operand order bahut important

Java code

```
import java.util.*;

class Solution {
    public int evalRPN(String[] tokens) {
        Deque<Integer> st = new ArrayDeque<>();

        for (String token : tokens) {
            if (token.equals("+") || token.equals("-") ||
                token.equals("*") || token.equals("/")) {
                int b = st.pop();
                int a = st.pop();

                switch (token) {
                    case "+": st.push(a + b); break;
                    case "-": st.push(a - b); break;
                    case "*": st.push(a * b); break;
                    case "/": st.push(a / b); break; // Java truncates toward zero
                }
            } else {
                st.push(Integer.parseInt(token));
            }
        }
        return st.pop();
    }
}
```

Example ` ["2", "1", "+", "3", "*"] `: 2 + 1 = 3; phir 3 × 3 = 9. Operator par pehle `b` pop, phir `a`; subtraction/division mein order ulta mat karna.

7. Queue fundamentals

Queue FIFO follow karti hai. Java mein `Queue` interface use karo; commonly `ArrayDeque` ya `LinkedList` implementation hoti hai.

Java code

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        Queue<Integer> q = new ArrayDeque<>();

        q.offer(10); // rear
        q.offer(20);
        q.offer(30);

        System.out.println(q.peek()); // front = 10
        System.out.println(q.poll()); // removes 10
        System.out.println(q.poll()); // removes 20
        System.out.println(q.size()); // 1
    }
}
```

Queue operations: offer/poll/peek prefer karo

Method	Meaning	Empty / full behavior
offer(x)	Add at rear	Capacity reject ho to false
poll()	Remove + return front	Empty par null
peek()	Read front, no removal	Empty par null
add(x)	Add at rear	Failure par exception
remove()	Remove front	Empty par exception
element()	Read front	Empty par exception

Interview code mein `offer`, `poll`, `peek` safer aur expressive hain. `ArrayDeque` null elements accept nahi karta; isliye `poll()` ka null empty signal ke liye useful hai.

Queue kab use karni hai?

- BFS in graph/tree: current level ke nodes process karke next level add karo.
- Task scheduling: arrival order preserve karna ho.
- Producer-consumer / buffering: items arrival order mein process karne ho.
- Simulation: line, printer jobs, request processing.

8. Queue implementation using array: Circular Queue

Simple array queue mein front se remove karte rahoge to khaali slots waste ho sakte hain. Circular queue indices ko modulo capacity se wrap karti hai.

Common formulas: `rear = (rear + 1) % capacity`; `front = (front + 1) % capacity`. Is implementation mein `size` store karne se full/empty distinguish karna easy hota hai.

Java code

```
class MyCircularQueue {
    private int[] data;
    private int front = 0;
    private int size = 0;

    public MyCircularQueue(int k) {
        data = new int[k];
    }

    public boolean enQueue(int value) {
        if (isFull()) return false;
        int rear = (front + size) % data.length;
        data[rear] = value;
        size++;
        return true;
    }

    public boolean deQueue() {
        if (isEmpty()) return false;
        front = (front + 1) % data.length;
        size--;
        return true;
    }

    public int Front() {
        return isEmpty() ? -1 : data[front];
    }

    public int Rear() {
        if (isEmpty()) return -1;
        int idx = (front + size - 1) % data.length;
        return data[idx];
    }

    public boolean isEmpty() { return size == 0; }
    public boolean isFull() { return size == data.length; }
}
```

Complexity: enqueue/dequeue/front/rear O(1), space O(k). Assumption: capacity `k > 0`; agar general-purpose class bana rahe ho to constructor mein validate karo.

Circular queue dry run: capacity 3

Operation	front	size	Logical queue
enqueue 7	0	1	[7]
enqueue 8	0	2	[7, 8]
enqueue 9	0	3	[7, 8, 9] full
dequeue	1	2	[8, 9]
enqueue 10	1	3	[8, 9, 10] (wrap-around)

9. Queue using two Stacks

Classic interview question: Stack available hai, Queue implement karo. Input stack mein enqueue; dequeue ke liye output stack. Jab output empty ho tab input ke saare elements transfer karo.

Java code

```
import java.util.*;

class MyQueue {
    private Deque<Integer> in = new ArrayDeque<>();
    private Deque<Integer> out = new ArrayDeque<>();

    public void push(int x) { // enqueue
        in.push(x);
    }

    public int pop() { // dequeue
        moveIfNeeded();
        if (out.isEmpty()) throw new NoSuchElementException();
        return out.pop();
    }

    public int peek() {
        moveIfNeeded();
        if (out.isEmpty()) throw new NoSuchElementException();
        return out.peek();
    }

    public boolean empty() {
        return in.isEmpty() && out.isEmpty();
    }

    private void moveIfNeeded() {
        if (out.isEmpty()) {
            while (!in.isEmpty()) out.push(in.pop());
        }
    }
}
```

Amortized complexity: each element maximum ek baar `in` se `out` transfer hota hai. Enqueue O(1); dequeue/peek amortized O(1), worst-case ek operation O(n); space O(n).

Dry run

- push 1, push 2, push 3 → in top-to-bottom: 3, 2, 1; out empty.
- pop → transfer: out top-to-bottom 1, 2, 3; pop returns 1.
- push 4 → in contains 4; out still has 2, 3.
- pop → out directly returns 2; transfer abhi nahi hoga.

10. Queue vs PriorityQueue

Normal Queue FIFO hai. `PriorityQueue` FIFO nahi: smallest (default min-heap) element pehle nikalta hai, chahe baad mein insert hua ho.

Java code

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        PriorityQueue<Integer> minHeap = new PriorityQueue<>();
```

```
        minHeap.offer(30);
        minHeap.offer(10);
        minHeap.offer(20);

        System.out.println(minHeap.poll()); // 10
        System.out.println(minHeap.poll()); // 20

        PriorityQueue<Integer> maxHeap =
            new PriorityQueue<>(Collections.reverseOrder());
        maxHeap.offer(30);
        maxHeap.offer(10);
        System.out.println(maxHeap.poll()); // 30
    }
}
```

Structure	Removal order	Typical operations
Queue	Oldest inserted first	offer/poll O(1) typical
PriorityQueue min-heap	Smallest priority first	offer/poll O(log n), peek O(1)
PriorityQueue max-heap	Largest priority first	offer/poll O(log n), peek O(1)

Use PriorityQueue for top K, kth largest/smallest, merge K sorted lists, Dijkstra, scheduling by priority. Use normal Queue for BFS and FIFO.

11. BFS: Queue ka sabse important interview use

Problem signal: unweighted graph mein shortest path / minimum number of edges; tree level-order traversal; nearest state by number of moves. BFS queue se level-by-level explore karta hai.

Java code

```
import java.util.*;

class Solution {
    public int[] bfs(int n, List<List<Integer>> graph, int source) {
        int[] dist = new int[n];
        Arrays.fill(dist, -1);

        Queue<Integer> q = new ArrayDeque<>();
        q.offer(source);
        dist[source] = 0; // visited at enqueue time

        while (!q.isEmpty()) {
            int node = q.poll();

            for (int nei : graph.get(node)) {
                if (dist[nei] == -1) {
                    dist[nei] = dist[node] + 1;
                    q.offer(nei);
                }
            }
        }
        return dist;
    }
}
```

Important: node ko visited mark enqueue karte waqt karo, dequeue par nahi. Warna same node multiple times queue mein ja sakta hai.

Complexity: adjacency-list graph par O(V + E) time, O(V) extra space.

Binary tree level-order traversal

Java code

```
import java.util.*;

class TreeNode {
    int val;
    TreeNode left, right;
    TreeNode(int val) { this.val = val; }
}

class Solution {
    public List<List<Integer>> levelOrder(TreeNode root) {
        List<List<Integer>> ans = new ArrayList<>();
        if (root == null) return ans;

        Queue<TreeNode> q = new ArrayDeque<>();
        q.offer(root);
```

```
        while (!q.isEmpty()) {
            int levelSize = q.size();
            List<Integer> level = new ArrayList<>();

            for (int i = 0; i < levelSize; i++) {
                TreeNode node = q.poll();
                level.add(node.val);

                if (node.left != null) q.offer(node.left);
                if (node.right != null) q.offer(node.right);
            }
            ans.add(level);
        }
        return ans;
    }
}
```

`levelSize = q.size()` ko loop se pehle store karna important hai. Isse current level ke nodes hi process honge; children next level ke liye queue mein rahenge.

12. Deque fundamentals

Deque (Double-Ended Queue) mein front aur rear dono taraf insertion/removal possible hai. Java mein `Deque` interface aur `ArrayDeque` implementation common choice hai.

Java code

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        Deque<Integer> dq = new ArrayDeque<>();

        dq.addFirst(20); // [20]
        dq.addLast(30);  // [20, 30]
        dq.offerFirst(10); // [10, 20, 30]
        dq.offerLast(40);  // [10, 20, 30, 40]

        System.out.println(dq.peekFirst()); // 10
        System.out.println(dq.peekLast());  // 40

        dq.pollFirst(); // removes 10
        dq.pollLast();  // removes 40
    }
}
```

Deque method cheat sheet

Front end	Rear end
addFirst(x) / offerFirst(x)	addLast(x) / offerLast(x)
removeFirst() / pollFirst()	removeLast() / pollLast()
getFirst() / peekFirst()	getLast() / peekLast()

`offer` / `poll` / `peek` variants generally return false/null on failure/empty; `add` / `remove` / `get` variants can throw exceptions. `ArrayDeque` null store nahi karta.

Deque ko Stack aur Queue ki tarah use karna

Java code

```
Deque<Integer> stack = new ArrayDeque<>();
stack.push(1);       // addFirst
stack.push(2);
int top = stack.pop(); // removeFirst

Deque<Integer> queue = new ArrayDeque<>();
queue.offerLast(1);
queue.offerLast(2);
int first = queue.pollFirst();
```

Same `ArrayDeque` ko stack-style (`push/pop/peek`) ya queue-style (`offerLast/pollFirst/peekFirst`) use kar sakte ho. Ek hi object mein dono conventions mix karne se confusion hota hai.

13. Monotonic Deque: Sliding Window Maximum

Problem: array aur window size k diya hai. Har contiguous window ka maximum return karo.

Why Deque? Humein window ke candidate maximum chahiye. Deque mein indices rakho; values decreasing order mein maintain karo. Front current window ka maximum hoga.

Rules: (1) front se out-of-window index remove. (2) Back se chhoti ya equal values remove, kyunki current value unse newer aur at least as large hai. (3) Current index add. (4) Window complete ho to front answer.

Java code

```
import java.util.*;

class Solution {
    public int[] maxSlidingWindow(int[] nums, int k) {
        if (nums == null || k <= 0 || k > nums.length) {
            return new int[0];
        }

        int n = nums.length;
        int[] ans = new int[n - k + 1];
        Deque<Integer> dq = new ArrayDeque<>(); // indices
        int write = 0;

        for (int i = 0; i < n; i++) {
            // Remove indices outside current window.
            while (!dq.isEmpty() && dq.peekFirst() <= i - k) {
                dq.pollFirst();
            }

            // Keep values decreasing from front to back.
            while (!dq.isEmpty() &&
                   nums[dq.peekLast()] <= nums[i]) {
                dq.pollLast();
            }

            dq.offerLast(i);

            if (i >= k - 1) {
                ans[write++] = nums[dq.peekFirst()];
            }
        }
        return ans;
    }
}
```

Dry run: nums = [1, 3, -1, -3, 5], k = 3

i	Current window	Deque indices (front → back)	Maximum
0	incomplete	[0]	—
1	incomplete	[1]	—
2	[1, 3, -1]	[1, 2]	3
3	[3, -1, -3]	[1, 2, 3]	3
4	[-1, -3, 5]	[4]	5

Correct output: [3, 3, 5]. **Complexity:** O(n) time, O(k) space. Har index deque mein maximum ek baar enter aur ek baar exit karta hai.

Critical detail: Deque mein values nahi, indices store karo. Indices se pata chalta hai kaunsa element window se bahar ho gaya.

14. Sliding Window Minimum / Maximum minus Minimum

Minimum ke liye symmetric monotonic increasing deque use karo: back se larger/equal values remove, front minimum deta hai. Agar har window ka max-min nikalna ho to ek decreasing deque (max) aur ek increasing deque (min) maintain karo.

Java code

```
import java.util.*;

class Solution {
    public int[] minSlidingWindow(int[] a, int k) {
        if (a == null || k <= 0 || k > a.length) return new int[0];

        int[] ans = new int[a.length - k + 1];
        Deque<Integer> dq = new ArrayDeque<>();
        int p = 0;
```

```
        for (int i = 0; i < a.length; i++) {
            while (!dq.isEmpty() && dq.peekFirst() <= i - k)
                dq.pollFirst();

            while (!dq.isEmpty() && a[dq.peekLast()] >= a[i])
                dq.pollLast();

            dq.offerLast(i);

            if (i >= k - 1) ans[p++] = a[dq.peekFirst()];
        }
        return ans;
    }
}
```

Longest subarray where max - min <= limit

Java code

```
import java.util.*;

class Solution {
    public int longestSubarray(int[] nums, int limit) {
        Deque<Integer> maxDq = new ArrayDeque<>(); // decreasing values
        Deque<Integer> minDq = new ArrayDeque<>(); // increasing values

        int left = 0, best = 0;

        for (int right = 0; right < nums.length; right++) {
            while (!maxDq.isEmpty() &&
                   nums[maxDq.peekLast()] < nums[right])
                maxDq.pollLast();
            maxDq.offerLast(right);

            while (!minDq.isEmpty() &&
                   nums[minDq.peekLast()] > nums[right])
                minDq.pollLast();
            minDq.offerLast(right);

            while ((long) nums[maxDq.peekFirst()] -
                   nums[minDq.peekFirst()] > limit) {
                if (maxDq.peekFirst() == left) maxDq.pollFirst();
                if (minDq.peekFirst() == left) minDq.pollFirst();
                left++;
            }

            best = Math.max(best, right - left + 1);
        }
        return best;
    }
}
```

Use `long` in subtraction to avoid integer overflow. Complexity O(n) time and O(n) worst-case space.

15. 0-1 BFS: Deque ka advanced graph use

Graph edges ka weight sirf 0 ya 1 ho to Dijkstra ke heap ki jagah 0-1 BFS use kar sakte hain. Weight 0 edge se milne wala node front par; weight 1 edge se milne wala node back par add hota hai.

Java code

```
import java.util.*;

class Edge {
    int to, weight;
    Edge(int to, int weight) { this.to = to; this.weight = weight; }
}

class Solution {
    public int[] zeroOneBfs(List<List<Edge>> graph, int src) {
        int n = graph.size();
        int INF = 1_000_000_000;
        int[] dist = new int[n];
        Arrays.fill(dist, INF);
        dist[src] = 0;

        Deque<Integer> dq = new ArrayDeque<>();
        dq.offerFirst(src);

        while (!dq.isEmpty()) {
            int u = dq.pollFirst();

            for (Edge e : graph.get(u)) {
                int nd = dist[u] + e.weight;
                if (nd < dist[e.to]) {
```

```
                    dist[e.to] = nd;
                    if (e.weight == 0) dq.offerFirst(e.to);
                    else dq.offerLast(e.to);
                }
            }
        }
        return dist;
    }
}
```

Complexity: O(V + E) for a 0/1 weighted graph with standard 0-1 BFS assumptions. Is technique ko arbitrary non-negative weights ke liye use mat karo; wahan Dijkstra chahiye.

16. Other important interview applications

Asteroid Collision — Stack

Positive asteroid right move karta hai, negative left. Collision tabhi possible jab stack top positive ho aur incoming negative. Smaller magnitude explode; equal magnitude dono explode.

Java code

```
import java.util.*;

class Solution {
    public int[] asteroidCollision(int[] asteroids) {
        Deque<Integer> st = new ArrayDeque<>();

        for (int x : asteroids) {
            boolean alive = true;

            while (alive && x < 0 && !st.isEmpty() && st.peek() > 0) {
                int top = st.peek();

                if (top < -x) st.pop();
                else if (top == -x) {
                    st.pop();
                    alive = false;
                } else {
                    alive = false;
                }
            }

            if (alive) st.push(x);
        }

        int[] ans = new int[st.size()];
        for (int i = ans.length - 1; i >= 0; i--) ans[i] = st.pop();
        return ans;
    }
}
```

Assumption: asteroid magnitude input range aisa ho ki `-x` overflow na kare (typical problem constraints).

Remove K Digits — Monotonic Stack

Java code

```
import java.util.*;

class Solution {
    public String removeKdigits(String num, int k) {
        Deque<Character> st = new ArrayDeque<>();

        for (char digit : num.toCharArray()) {
            while (k > 0 && !st.isEmpty() && st.peek() > digit) {
                st.pop();
                k--;
            }
            st.push(digit);
        }

        while (k > 0 && !st.isEmpty()) {
            st.pop();
            k--;
        }

        StringBuilder sb = new StringBuilder();
        while (!st.isEmpty()) sb.append(st.removeLast());

        // Remove leading zeros.
        int i = 0;
        while (i < sb.length() && sb.charAt(i) == '0') i++;
```

```
        String result = sb.substring(i);
        return result.isEmpty() ? "0" : result;
    }
}
```

Idea: smaller digit ko earlier position par laane ke liye previous larger digits remove karo. Remaining k ko end se remove karo. Time O(n), space O(n).

Decode String — Nested repetition

Java code

```
import java.util.*;

class Solution {
    public String decodeString(String s) {
        Deque<Integer> counts = new ArrayDeque<>();
        Deque<StringBuilder> prefixes = new ArrayDeque<>();
        StringBuilder current = new StringBuilder();
        int number = 0;

        for (char ch : s.toCharArray()) {
            if (Character.isDigit(ch)) {
                number = number * 10 + (ch - '0');
            } else if (ch == '[') {
                counts.push(number);
                prefixes.push(current);
                number = 0;
                current = new StringBuilder();
            } else if (ch == ']') {
                int repeat = counts.pop();
                StringBuilder parent = prefixes.pop();
                for (int i = 0; i < repeat; i++) parent.append(current);
                current = parent;
            } else {
                current.append(ch);
            }
        }
        return current.toString();
    }
}
```

Example `3[a2[c]]` → `accaccacc`. Count stack multi-digit numbers handle karta hai; string stack nested prefix restore karta hai.

17. Complexity master table

Operation / structure	Time	Space note
ArrayDeque push/pop/peek	O(1) amortized / O(1)	O(n) total storage
Queue offer/poll/peek (ArrayDeque)	O(1) amortized / O(1)	O(n) total storage
Deque add/remove at ends	O(1) amortized	O(n) total storage
Balanced brackets	O(n)	O(n)
Next greater element	O(n)	O(n)
Largest histogram rectangle	O(n)	O(n)
BFS adjacency list	O(V + E)	O(V)
Sliding window max/min	O(n)	O(k)
0-1 BFS	O(V + E)	O(V)

Amortized O(1) ka matlab: individual operation kabhi resize ke kaaran costly ho sakta hai, lekin bahut saare operations ka average constant hota hai.

18. Common mistakes — interview mein avoid karo

- Stack mein `pop()` se pehle empty check bhoolna.
- Parentheses problem mein bracket type match na karna.
- Next Greater Element mein values push karna jab duplicate values hain; indices zyada safe hain.

- Monotonic stack mein `>` vs `>=` ka decision problem ke “strictly greater” ya “greater or equal” wording ke hisaab se na lena.
- BFS mein visited ko enqueue karte waqt mark na karna.
- Level-order traversal mein `levelSize` ko loop ke andar recalculate karna.
- Sliding window deque mein expired indices remove na karna.
- Deque mein index ki jagah value store karna aur phir duplicate values ke expiry mein confuse hona.
- PriorityQueue ko FIFO Queue samajhna.
- Subtraction `max - min` mein integer overflow ignore karna.
- Java `ArrayDeque` mein null insert karne ki koshish karna.
- `Stack` legacy class ko bina reason default choose karna.

Strict vs non-strict comparison

Monotonic stack/deque mein comparison sabse common bug hai. Example: Next strictly greater element ke liye current ko previous se strictly bada hona chahiye (`current > previous`). Equal value answer nahi hai. Lekin duplicate handling ke liye previous smaller/equal jaisi problem mein comparison alag hoga. Problem statement ko exact read karo.

19. Java templates — yaad rakhne layak

Stack template

Java code

```
Deque<Integer> st = new ArrayDeque<>();
for (int x : nums) {
    while (!st.isEmpty() && shouldRemove(st.peek(), x)) {
        st.pop();
    }
    st.push(x);
}
```

Queue / BFS template

Java code

```
Queue<Integer> q = new ArrayDeque<>();
boolean[] visited = new boolean[n];

q.offer(src);
visited[src] = true;

while (!q.isEmpty()) {
    int u = q.poll();
    for (int v : graph.get(u)) {
        if (!visited[v]) {
            visited[v] = true;
            q.offer(v);
        }
    }
}
```

Deque sliding window template

Java code

```
Deque<Integer> dq = new ArrayDeque<>(); // indices
for (int right = 0; right < n; right++) {
    while (!dq.isEmpty() && dq.peekFirst() < windowStart)
        dq.pollFirst();

    while (!dq.isEmpty() && nums[dq.peekLast()] <= nums[right])
        dq.pollLast();

    dq.offerLast(right);
    // front index is current window maximum
}
```

20. Interview problem checklist

Level	Problem / topic	Pattern to learn
Easy	Valid Parentheses	Stack matching
Easy	Implement Stack / Queue	API + basic operations
Easy	Implement Queue using Stacks	Two-stack transfer
Easy	Implement Stack using Queues	Rotation / two queues
Easy	Backspace String Compare	Stack or reverse scan
Medium	Min Stack	Auxiliary minimum
Medium	Next Greater Element I/II	Monotonic stack
Medium	Daily Temperatures	Next greater + distance
Medium	Evaluate Reverse Polish Notation	Operand stack
Medium	Asteroid Collision	Simulation stack
Medium	Decode String	Nested stacks
Medium	Simplify Path	Path stack
Medium	Sliding Window Maximum	Monotonic deque
Medium	Design Circular Queue	Circular array
Medium	Number of Recent Calls	Queue / time window
Medium	Binary Tree Level Order	BFS queue
Medium	Rotting Oranges	Multi-source BFS
Hard	Largest Rectangle in Histogram	Monotonic stack
Hard	Maximal Rectangle	Histogram + stack
Hard	Shortest Subarray with Sum at Least K	Prefix sum + monotonic deque
Hard	Longest Continuous Subarray with Abs Diff Limit	Two monotonic deques
Hard	Shortest Path in Binary Matrix	BFS queue
Advanced	0-1 BFS	Deque + weighted graph

21. 14-day practice plan

Day	Focus	Target
1	Stack API + Valid Parentheses	3 problems
2	Min Stack + RPN	2 problems
3	Next Greater / Smaller	3 problems
4	Daily Temperatures + Stock Span	2 problems
5	Histogram + Maximal Rectangle	2 problems
6	Queue API + Circular Queue	2 problems
7	Queue using Stacks / Stack using Queues	2 problems
8	BFS basics + tree level order	3 problems
9	Rotting Oranges + shortest path grid	2 problems
10	Deque operations + sliding max	2 problems
11	Sliding min + max-min limit	2 problems
12	Decode String + Asteroid Collision	2 problems
13	0-1 BFS + advanced deque	1–2 problems
14	Revision: solve without notes	5 mixed problems

Daily method: pehle 10–15 minute brute force socho, phir data structure ka reason likho, phir code bina copy kiye likho, phir edge cases aur complexity explain karo.

22. Self-test: khud se answer do

- 1. Stack ka order kya hota hai? Queue ka?
- 2. Java mein Stack ke liye `ArrayDeque` kyun prefer karte hain?
- 3. `peek()` aur `pop()` mein difference?
- 4. BFS mein visited enqueue ke waqt mark kyun karte hain?
- 5. Next Greater Element O(n) kaise hai jab nested while loop hai?
- 6. Sliding window maximum mein deque indices kyun rakhta hai?
- 7. PriorityQueue FIFO kyun nahi hai?
- 8. Circular queue mein modulo kyun use hota hai?
- 9. 0-1 BFS mein weight 0 front aur weight 1 back par kyun?
- 10. Histogram mein popped bar ki width ka formula kya hai?

Answer key

- 1. Stack LIFO; Queue FIFO.
- 2. Modern Deque API, efficient end operations; Stack legacy class hai.
- 3. `peek` dekhta hai; `pop` remove karke return karta hai.
- 4. Duplicate enqueues / repeated processing se bachne ke liye.
- 5. Har index maximum ek baar push aur ek baar pop hota hai (amortized analysis).
- 6. Expiry/window boundary identify karne ke liye.
- 7. Heap priority ke according remove karta hai.
- 8. Array index ko end par pahunchne ke baad start par wrap karne ke liye.
- 9. 0-cost transition ko current distance layer mein pehle process karna hota hai.

• 10. `rightIndex - leftSmallerIndex - 1`.

23. Final one-page revision sheet

Clue in question	Think of	Core template
Last added, first removed	Stack	push / pop / peek
Nested brackets / undo	Stack	push open; match on close
Next greater / smaller	Monotonic Stack	pop unresolved indices
Largest rectangle	Monotonic Stack	boundaries around popped bar
First come, first served	Queue	offer / poll
Minimum edges in unweighted graph	Queue + BFS	mark visited on enqueue
Process each tree level	Queue	capture levelSize
Both ends needed	Deque	addFirst/addLast, pollFirst/pollLast
Window maximum/minimum	Monotonic Deque	indices + expire + maintain order
Only edge weights 0 and 1	Deque + 0-1 BFS	0 front, 1 back
Smallest/largest priority first	PriorityQueue	heap, not FIFO

Final advice: Stack, Queue aur Deque ko alag-alag definitions ke roop mein mat padho. Inke patterns master karo: matching, monotonic order, BFS, circular indexing aur sliding window. Interview mein wahi repeat hote hain.